VGU COLLEGE · JAVA

Java Collections Framework

Module III — An Overview

BEFORE WE START

Before we learn each collection, we answer three questions:
What problem do collections solve? What is a collection? Which one do I use?

Contents

01 Where We Left Module II 2
02 The Problem: Arrays Are Not Enough 2
03 The Idea 2
04 Definitions 3
05 What the Framework Gives Us 3
06 Array vs Collection 3
07 Same Job, Two Ways 4
08 The Family Tree 4
09 Four Families, Four College Jobs 5
10 Interface vs Class 5
11 What Is `<Student>`? 5
12 Collections Store Objects Only 6
13 Methods Every Collection Has 6
14 `Collection` vs `Collections` 6
15 Which One Do I Use? 7
16 Important Points 7
17 Our Journey in Module III 7
18 Quick Check 8
19 Remember 8

Where We Left Module II

Our VGU College Portal kept every member in an **array**:

```
CollegeMember[] members = new CollegeMember[60];
int size = 0;
```

Roll no	Name	Total
101	Hita	438
102	Sakshi	244
103	Somya	465
104	Ritika	412
105	Disha	438

It worked... until the college grew.

The Problem: Arrays Are Not Enough

The college says...	With an array
"61st student took admission"	❌ Array is full. Size is fixed.
"Sakshi moved to section B"	❌ Shift every student after her by hand
"Ritika joined late, put her at roll 104"	❌ Shift everyone to make space
"Is Somya in this section?"	❌ Write a loop
"Show me roll no 103"	❌ Write a loop
"Give me the merit list"	❌ Write our own sorting code
"Hita signed up twice for the workshop"	❌ Write code to stop duplicates

> **IMPORTANT**
> We are writing **storage code** again and again, instead of **college code**.

The Idea

What if Java already had **ready-made containers** that:

- grow and shrink by themselves
- add, remove, search and sort with **one method call**
- stop duplicates for us
- find a student by roll number **without a loop**

👉 **That is the Collections Framework.**

Definitions

DEFINITION

Collection

An object that stores a group of objects (called **elements**) as a single unit.

DEFINITION

Framework

A ready-made set of interfaces and classes that gives a standard way to solve a problem.

DEFINITION

Collections Framework

A set of interfaces and classes in the `java.util` package used to **store** and **work on** a group of objects (add, remove, search, sort).

What the Framework Gives Us

Part	Meaning	Examples
Interfaces	WHAT a container can do	`List`, `Set`, `Queue`, `Map`
Classes	HOW it is done (ready to use)	`ArrayList`, `HashSet`, `HashMap`
Algorithms	Ready-made helper methods	`Collections.sort()`, `Collections.max()`

NOTE

All of them live in `java.util`.

Array vs Collection

	Array	Collection
Size	Fixed when created	Grows and shrinks by itself
Stores	Primitives and objects	**Objects only**
Ready-made methods	None (only `length`)	`add`, `remove`, `contains`, `size` …
Insert in the middle	Shift by hand	`add(index, element)`
Search / sort	Write it yourself	One method call
Print all	Loop needed	`System.out.println(list)`

NOTE

Array is still fine when the size **never changes** (a student always has 5 marks).

Same Job, Two Ways

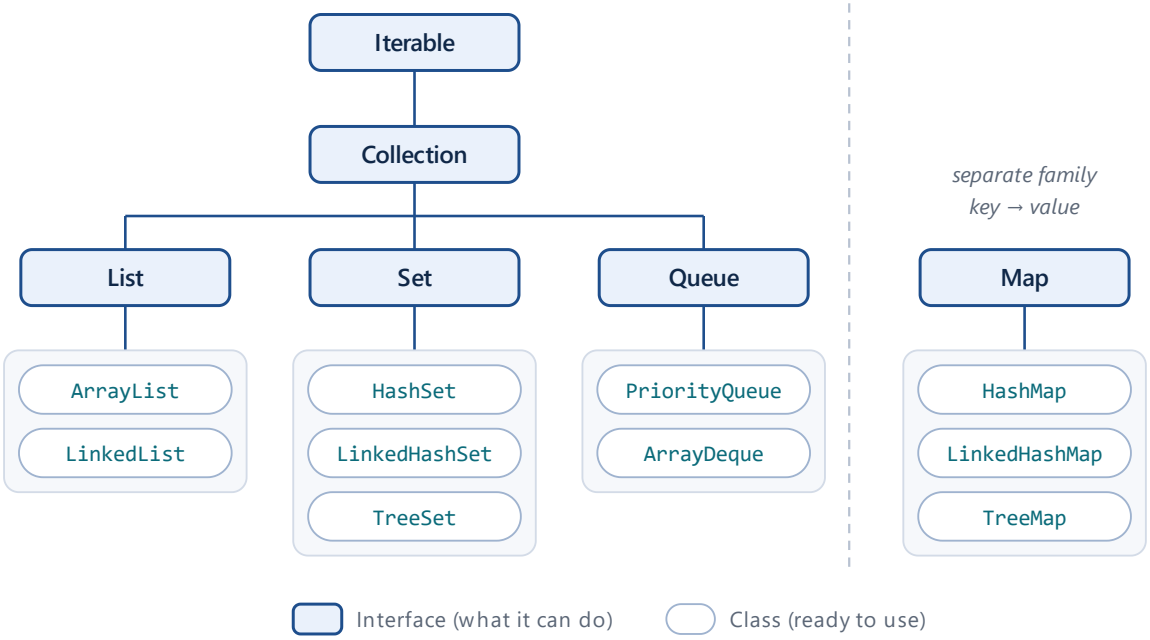
EXAMPLE · ARRAY

```
Student[] section = new Student[3];
section[0] = new Student(101, "Hita", 438);
section[1] = new Student(102, "Sakshi", 244);
section[2] = new Student(103, "Somya", 465);
section[3] = new Student(104, "Ritika", 412);   // ❌ ArrayIndexOutOfBoundsException
```

EXAMPLE · COLLECTION

```
List<Student> section = new ArrayList<>();
section.add(new Student(101, "Hita", 438));
section.add(new Student(102, "Sakshi", 244));
section.add(new Student(103, "Somya", 465));
section.add(new Student(104, "Ritika", 412));     // ✅ list grows
section.remove(1);                                // ✅ Sakshi leaves, no shifting
System.out.println(section);                      // ✅ prints everyone
```

The Family Tree



- `Map` is **not** a `Collection`: it stores **pairs**, not single elements.

Four Families, Four College Jobs

Family	Rule	VGU College example
List	Ordered, index-based, duplicates allowed	Section register in roll order
Set	**No duplicates**	Workshop sign-up: each name once
Queue	**First in, first out**	Line at the fee counter
Map	**Key → value**, unique keys	Roll no → student, find 103 directly

> **TIP**
> Learn the **rule** of each family. The class names come later.

Interface vs Class

```
List<Student> section = new ArrayList<>();
//  interface              class
//  (WHAT it can do)       (HOW it does it)
```

- Write the **interface** on the left.
- Want a `LinkedList` later? Change **one word** on the right.
- Same idea as Module II: `Payment mode = new UpiPayment(...)`

What Is `<Student>`?

```
List<Student> section = new ArrayList<>();
```

- `<Student>` tells Java: **this list holds only Students**.
- Put a `String` in by mistake → **compile error** (caught early).
- Read from it → no cast needed.

> **NOTE**
> This is called **Generics** (topic 3.10).

Collections Store Objects Only

```
List<Integer> marks = new ArrayList<>();
marks.add(85);          // int → Integer   (autoboxing)
int m = marks.get(0);   // Integer → int   (unboxing)

// List<int> x;         // ✗ not allowed
```

Primitive	Use in a collection
`int`	`Integer`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

Methods Every Collection Has

Method	Does
`add(x)`	Add an element
`remove(x)`	Remove an element
`contains(x)`	Is it there?
`size()`	How many?
`isEmpty()`	Is it empty?
`clear()`	Remove everything

TIP

Learn these once, use them on **every** List, Set and Queue.

`Collection` vs `Collections`

`Collection`	`Collections`
An **interface**	A **utility class**
Parent of List, Set, Queue	Only `static` helper methods
`section.add(x)`	`Collections.sort(section)`

WATCH OUT

One letter, two very different things.

Which One Do I Use?

```
Size never changes?              → Array
Need order + duplicates OK?      → List    (ArrayList)
Need no duplicates?              → Set     (HashSet)
First come, first served?        → Queue   (LinkedList / ArrayDeque)
Find by a key (roll no)?         → Map     (HashMap)
```

Need	Pick
Keep order too	`LinkedHashSet`, `LinkedHashMap`
Keep it sorted	`TreeSet`, `TreeMap`

Important Points

KEY POINTS

- A collection stores a **group of objects** as one unit.
- Collections **grow and shrink** by themselves; arrays don't.
- Collections store **objects only** (`Integer`, not `int`).
- Four families: **List, Set, Queue, Map**. `Map` is not a `Collection`.
- Interface on the left, class on the right: `List<Student> s = new ArrayList<>();`
- `<Type>` keeps the collection safe (Generics).
- `Collection` = interface, `Collections` = helper class.
- Everything is in `java.util`.

Our Journey in Module III

Topic	Question it answers
3.2 Collection interface	What can every collection do?
3.3 List	How do I keep students in order?
3.4 Set	How do I stop duplicates?
3.5 Queue / Deque	How do I serve first come, first served?
3.6 Map	How do I find a student by roll no?
3.7 Iterators	How do I remove while looping?
3.8 Comparable / Comparator	How do I make a merit list?
3.9 Collections class	What ready-made helpers exist?
3.10 Generics	How does `<Student>` keep us safe?
3.11 Nested classes	Classes inside classes
3.12 Callbacks	How do objects react when something happens?

Quick Check

YOUR TURN

Which collection would you use?

1. The list of all students of CSE-A, in admission order.
2. Students who signed up for the Java workshop (no one twice).
3. Students standing at the fee counter.
4. Find a student's details from the roll number.
5. The 5 subject marks of one student.

ANSWERS

1. **List** (`ArrayList`)
2. **Set** (`LinkedHashSet` to keep sign-up order)
3. **Queue**
4. **Map** (`HashMap<Integer, Student>`)
5. **Array** (size is always 5)

Remember

KEY TAKEAWAY

Array → fixed box.
Collection → smart container that grows, searches, sorts and stops duplicates for you.

Choose the container by the job.